

Abdul Basit Sajid

Backend / Infrastructure Engineer

basit@basit.engineer | +92 324 5717425 | Remote — Pakistan (UTC+5)
basit.engineer | github.com/abd-ulbasit | linkedin.com/in/abd-ulbasit

SUMMARY

Backend and infrastructure engineer, 3 years, remote from Pakistan for a UK company since 2023. Strongest work is capacity and latency engineering on a real-time GPU inference backend, and the cost analysis that argued against building the system that would have scaled it. Production experience is Python and Node on GCP/AWS; the Go, Kubernetes and Postgres-internals work below is open source.

EXPERIENCE

Software Engineer — EliteITTeam, Manchester UK (Remote)

Sept 2024 – Present

- Took ORB Vision, a real-time multi-model computer-vision backend, from a measured ~ 3 concurrent users to a proved capacity of 700 concurrent under load test — 99.4% frame delivery, 132 ms p50, \$0.0106 per user-hour.
- Cut per-frame inference 2,800 ms $\rightarrow$ 60 ms ($46\times$): four YOLO models onto CUDA, an $O(n^2)$ Python NMS loop replaced with `torchvision.ops.nms`, and per-detection GPU $\rightarrow$ CPU transfers batched into one copy.
- Cut p50 871 ms $\rightarrow$ 59 ms by removing a `broadcast=True` fan-out that made every response $O(N)$ in connected sockets and pinning each session to one container via an nginx hash — found in a 7-configuration sweep in which three of my own hypotheses were falsified, and the only change that moved the ceiling rather than the tail.
- Wrote the capacity decision document for a 60,000-user target: corrected an internal \$206k/mo projection to \$475–570k/mo from measured per-box capacity, established that GPUs are excluded from GCP's Flexible CUD, and recommended **not** building the interim GPU fleet — the adopted on-device path removes GPU inference cost entirely.
- Took a listings endpoint 2.5 s $\rightarrow$ 650 ms (74%) by collapsing six sequential MongoDB queries into one `$facet` aggregation.

Associate Software Engineer — EliteITTeam, Manchester UK (Remote)

Sept 2023 – Sept 2024

- Primary backend contributor on an edtech platform (107 of 165 commits), owning production migrations end to end: SMTP provider swap, staging environment rename, and backup procedures.
- Root-caused a Stripe webhook failure and repaired the affected production data with a dry-run pass, row checksums and a written rollback procedure before executing against live records.

OPEN-SOURCE INFRASTRUCTURE PROJECTS

pgbranch — copy-on-write Postgres branching (Go, OverlayFS, Docker)

github.com/abd-ulbasit/pgbranch

- Diagnosed why branching a 5 GiB database took 61.9 s and left a 5.05 GiB writable layer — a full copy inside a system whose premise is that it copies nothing. Postgres's pre-WAL-replay `SyncDataDirectory` opens every PGDATA file `O_RDWR`, and on OverlayFS an `O_RDWR` open of a lower-layer file forces whole-file copy-up.
- Proved the mechanism instead of arguing it: a single-variable control run with `recovery_init_sync_method=syncfs` ended at 16 KiB of writable layer, not 5.05 GiB. The one-flag fix takes create to 1.89 s / 33 MiB.
- Published benchmarks keep the pre-fix numbers that contradicted the project's thesis, and say why they are not comparable.

upgradescope — Kubernetes upgrade-readiness scanner (Go, client-go)

github.com/abd-ulbasit/upgradescope

- Collectors degrade instead of failing: a 401/403 from a managed control plane's metrics endpoint becomes a named unavailable capability, so the report states which signal was lost rather than reporting a clean scan.
- Argued against a controller-runtime reconciler: the verdict is a function of (inventory, knowledge base, target) and two of those three change with no Kubernetes event, so a watch-driven loop requeues on unrelated pod churn yet still misses the day a component's EOL passes.
- The API-lifecycle dataset is generated and CI-gated, plus a check that diffs `go list k8s.io/api/...` against the generator's imports so a new API group cannot silently pass.

goqueue — distributed message queue (Go)

github.com/abd-ulbasit/goqueue

- Found a bucket-drain race in a hierarchical timing wheel: an element pointer held across the callback unlock is silently invalidated by a concurrent cancel, truncating the traversal and stranding every remaining timer in that bucket for up to 7.76 days.
- Invisible to `-race` (every access really was under the mutex) and to the existing tests; wrote deterministic regression tests that fail against the old implementation, then fixed it.

forgepoint — ML-lifecycle platform, 11 Go services (gRPC, NATS JetStream)

github.com/abd-ulbasit/forgepoint

Saga compensation, event sourcing, transactional outbox and streaming drift detection on Kubernetes. A cancelled saga detaches its terminal write from the already-dead context, so the durable row can never contradict the API response.

WRITING, EDUCATION & CERTIFICATIONS

Bookstore Kubernetes Guide — 16 parts, 115 chapters, one application from a container to a production cluster. abd-ulbasit.github.io/bookstore-kubernetes-guide

BS Computer Science — NUST, Islamabad | **AWS Certified Solutions Architect – Associate**, July 2025

SKILLS

Languages: Go, Python, TypeScript, SQL, Rust | **Data:** PostgreSQL, MongoDB, Redis | **Cloud:** AWS, GCP

Infrastructure: Kubernetes, Docker, Helm, Terraform, Nginx, GitHub Actions, Prometheus, gRPC, NATS JetStream

Practices: Distributed systems, concurrency, load testing and capacity planning, cost modelling, production migrations